Array

Arrays power most real-world JavaScript data work. Mastering the core methods and loops is essential for exams, interviews, and everyday productivity. Use this Cheat-Sheet to refresh quickly.

Core actions

push(...items) → Add items to the end

```
const a = [1]; a.push(2,3); // [1,2,3]
```

pop() → Rremove and return the last item

```
const a = [1,2]; const last = a.pop(); // 2
```

unshift(...items) → Add items to the start

```
const a = [2]; a.unshift(0,1); // [0,1,2]
```

shift() → Remove and return the first item

```
const a = [1,2]; const first = a.shift(); // 1
```

splice(start, deleteCount, ...items) → Remove/insert at an index

```
const a = [1,2,3]; a.splice(1,1,'x'); //
[1,'x',3]
```

concat(...arrays) → Merge into a new array

```
[1].concat([2,3]); // [1,2,3]
```

Transform & Aggregate

map(fn) → Transform each element → new array

```
[1,2,3].map(x => x * 2); // [2,4,6]
```

filter(fn) → Keep elements that pass the test

```
[1,2,3,4].filter(x => x % 2 === 0); // [2,4]
```

reduce(fn, initial) → Combine into one value

```
[1,2,3].reduce((s,x) => s + x, 0); // 6
```

find(fn) → First matching element or `undefined`

```
[1,4,6].find(x => x > 3); // 4
```

Order & Combine

sort() → Sort elements as strings (alphabetical)

```
['b','a','c'].sort(); // ['a','b','c']
```

reverse() → Reverse the array in place

```
const a = [1,2,3]; a.reverse(); // [3,2,1]
```

join(separator) → Join elements into a string

```
['a','b','c'].join('-'); // 'a-b-c'
```

Cloning

spread `const copy = [...arr]`

slice `const copy = arr.slice()`

Array.from `const copy = Array.from(arr)`

concat `const copy = [].concat(arr)`

structuredClone `onst deep = structuredClone(arr)`

Note: spread/slice/Array.from/concat are shallow; `structuredClone` is deep (for structured data).

Array

Arrays power most real-world JavaScript data work. Mastering the core methods and loops is essential for exams, interviews, and everyday productivity. Use this Cheat-Sheet to refresh quickly.

Core actions

push(...items) → Add items to the end

```
const a = [1]; a.push(2,3); // [1,2,3]
```

pop() → Rremove and return the last item

```
const a = [1,2]; const last = a.pop(); // 2
```

unshift(...items) → Add items to the start

```
const a = [2]; a.unshift(0,1); // [0,1,2]
```

shift() → Remove and return the first item

```
const a = [1,2]; const first = a.shift(); // 1
```

splice(start, deleteCount, ...items) → Remove/insert at an index

```
const a = [1,2,3]; a.splice(1,1,'x'); //
[1,'x',3]
```

concat(...arrays) → Merge into a new array

```
[1].concat([2,3]); // [1,2,3]
```

Transform & Aggregate

map(fn) → Transform each element → new array

```
[1,2,3].map(x => x * 2); // [2,4,6]
```

filter(fn) → Keep elements that pass the test

```
[1,2,3,4].filter(x => x % 2 === 0); // [2,4]
```

reduce(fn, initial) → Combine into one value

```
[1,2,3].reduce((s,x) => s + x, 0); // 6
```

find(fn) → First matching element or `undefined`

```
[1,4,6].find(x => x > 3); // 4
```

Order & Combine

sort() → Sort elements as strings (alphabetical)

```
['b','a','c'].sort(); // ['a','b','c']
```

reverse() → Reverse the array in place

```
const a = [1,2,3]; a.reverse(); // [3,2,1]
```

join(separator) → Join elements into a string

```
['a','b','c'].join('-'); // 'a-b-c'
```

Cloning

spread `const copy = [...arr]`

slice `const copy = arr.slice()`

Array.from `const copy = Array.from(arr)`

concat `const copy = [].concat(arr)`

structuredClone `onst deep = structuredClone(arr)`

Note: spread/slice/Array.from/concat are shallow; `structuredClone` is deep (for structured data).